

Visual Inertial Navigation for a Small UAV Using Sparse and Dense Optical Flow

Fausto Fanin and Ju-Hyeon Hong

Abstract—There has been a recent surge in interest regarding small Unmanned Aerial Vehicles (UAV) that operate in indoor environments. A crucial part of such platforms is the navigation system that allows them to navigate in Global Navigation Satellite System (GNSS) denied environments. The two objectives of this work were to develop an Optical Flow (OF) based method for navigation and a high-fidelity simulation for testing and verification. This paper presents the Visual-Inertial Navigation System (VINS) developed that utilises sparse and dense Optical Flow patterns within an Extended Kalman Filter (EKF) framework for autonomous navigation in GNSS denied environments. The principal novelty is the use of dense Optical Flow measurements to estimate the velocity of the UAV.

I. INTRODUCTION

Small Unmanned Aerial Vehicles (UAV) capable of flying in indoor and urban environments have seen a very significant rise in popularity, both in the civilian and academic fields. Advances in almost every aspect of these UAV, such as batteries, Microelectromechanical systems (MEMS) sensors, computing performance, motor performance, etc. has enabled this mass proliferation. This has allowed these platforms to find many novel applications such as wing inspection [1], package delivery [2], and agriculture [3].

A core technology of these platforms is their navigation system and in the past few years considerable work has been done on the topic of Visual-Inertial Navigation Systems (VINSs). As these mobile robots cannot rely on traditional navigation systems such as Global Navigation Satellite System (GNSS), recent work has focused on achieving similar levels of precision utilising passive sensors such as Inertial Measurement Units (IMUs) and digital cameras. Advances for both these groups of sensors has made them smaller, less expensive and more accurate, which when coupled with the aforementioned work on VINSs, has rendered such systems suitably accurate for field applications such as agriculture [3], search and rescue, mapping, and surveying [1]. A reason VINSs are often preferred to other Simultaneous Localisation and Mapping (SLAM) solutions is their low weight and low cost. As the only two sensors needed are an IMU and a digital camera, such a solution often proves to be more weight and cost effective than a LiDAR-based solutions. This is especially the case for UAVs where the Maximum Take-Off Weight (MTOW) is a severe limiting factor.

Fausto Fanin and Ju-Hyeon Hong are with the School of Aerospace, Transport and Manufacturing, Cranfield University, Cranfield MK43 0AL, UK (email:faustofanin7@gmail.com and hong.ju.hyeon11@gmail.com)

An augmented VINS is developed that utilises dense optical flow patterns to introduce a new measurements for sensor fusion. The common framework for a VINS, that utilises a Kalman Filter (KF) to fuse IMU measurements with sparse optical flow patterns, is augmented by introducing an estimate of the UAV linear velocity derived from dense optical flow patterns. This novel approach of utilising a velocity measurement in the correction step of a Kalman Filter allows for discretization errors to be corrected at all levels of the kinodynamics.

This paper presents the development of this VINS and the high-fidelity simulation utilised for verification. First, an initial investigation into the feasibility of estimating velocity from dense optical flow patterns was preformed and later verified through experimental testing. Then, in order to validate the functionality and performance of the entire VINS a high fidelity simulation was developed including an accurate UAV aerodynamic and propulsion model, a detailed environment representative of an indoor office environments and an efficient computer vision pipeline to process the optical flow patterns. The results of this simulation are also presented and discussed.

II. OPTICAL FLOW

While a sparse optical flow pattern can be used to reliably track static points in the environment, as shown in [4], a dense optical flow pattern is more representative of the relative motion between the environment and the viewer.

A. Simulating Omnidirectional Optical Flow Patterns

An initial investigation into the feasibility of utilising an omni-directional optical flow pattern was simulated for a UAV with variable linear and angular velocity. This was done by calculating the relative velocities of a simulated environment and projecting these onto a sub-sphere, which was then unwrapped to achieve a standard optical flow image.

This is shown in Figure 1 and was expanded from the simplified spherical world assumption to function inside of a rectangular environment of variable dimensions.

B. Velocity Estimation

Our velocity estimation algorithm relies on segmenting the perceived optical flow pattern into different halves, each pair corresponding to a particular body-frame axis. As the optical flow field corresponds to a projection onto a sub-sphere, half of the optical flow pattern would correspond to

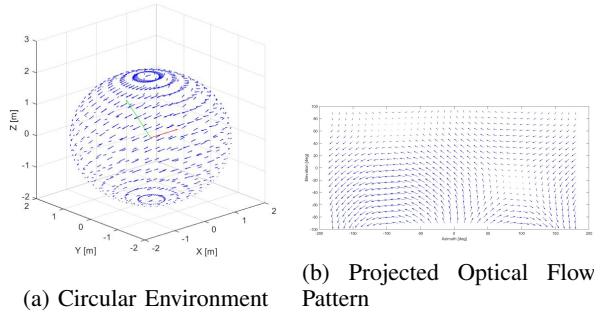


Fig. 1: Simulated Optical Flow

a hemisphere, which when averaged across both dimensions is used to calculate the overall velocity of the UAV.

Each hemisphere can be averaged into an x and y image-frame velocity, and as this average velocity is located at the centre of the image (and hemisphere). This location corresponds to the axis perpendicular to the bisecting plane. This estimated velocity from both hemispheres can be used to estimate the overall UAV velocity as shown in Figure 2.

In this example in Figure 2 the average velocities, v_l and v_r are represented by arrows depicting their magnitude. These two velocities can be each decomposed into two sub-velocities, of which one is equal in magnitude and direction and the other equal in magnitude but opposite in direction. The velocities equal in magnitude and direction, v_{lin} and v_{rlin} , can be found by averaging the original velocities v_l and v_r :

$$v_{rlin} = v_{lin} = \frac{1}{2} (v_l + v_r) \quad (1)$$

It then follows that the two velocities equal in magnitude but opposite in direction, v_{rang} and v_{lang} , can be found by computing the difference between the velocities from Equation 1 and the original velocities, v_l and v_r :

$$v_{rang} = v_l - v_{rlin} = -(v_{lang}) = -(v_r - v_{rlin}) \quad (2)$$

As insinuated by the suffixes, v_{lin} and v_{rlin} are equal to the linear velocity of the UAV along the axis bisecting the hemispheres. On the other hand v_{lang} and v_{rang} are not

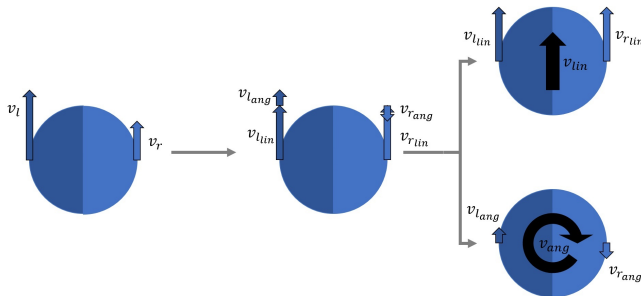


Fig. 2: 2-dimensional example of velocity estimation

directly the angular velocity but rather the linear component of the angular velocity.

$$v_{lin} = v_{lin} = v_{rlin} \quad (3)$$

$$v_{ang} \propto v_{lang} \quad \text{and} \quad v_{ang} \propto v_{rang}$$

Calculating the angular velocity would require a distance measurement between the UAV and the environment which, while possible to extract would greatly add to the complexity of the algorithm. As the IMU onboard a UAV provides a direct measurement of the angular velocity the need to estimate this is redundant and therefore was not further investigated.

III. NAVIGATION FILTER

This section provides an overview of the Extended Kalman Filter (EKF) framework utilised to recursively estimate the odometry of the UAV in this paper. First the continuous-time equations are utilised to describe the system before a detailed description of the discrete-time equations utilised in the actual implementation.

A. Continuous Time Domain

1) *State Propagation*: The state vector has a standard structure utilised in many Visual-Inertial Navigation filters:

$$\mathbf{x} = [\mathbf{B}_I^T \mathbf{q}^T \quad \mathbf{I}_I^T \mathbf{p}^T \quad \mathbf{I}_I^T \mathbf{v}^T \quad \mathbf{b}_a^T \quad \mathbf{b}_g^T]^T \quad (4)$$

where $\mathbf{B}_I^T \mathbf{q}^T$ is the unit quaternion representing the rotation from the inertial frame $\{I\}$ to the UAV body frame $\{B\}$; $\mathbf{I}_I^T \mathbf{p}^T$ and $\mathbf{I}_I^T \mathbf{v}^T$ are the position and velocity of the UAV in frame $\{I\}$, respectively, and $\mathbf{b}_a^T$ and $\mathbf{b}_g^T$ represent the accelerometer and gyroscope measurement biases, respectively.

Utilising the IMU motion dynamics [5] the continuous time dynamics of the state vector can be computed as:

$$\begin{aligned} \mathbf{B}_I^T \dot{\mathbf{q}}(t) &= \frac{1}{2} \boldsymbol{\Omega}(\omega(t)) \mathbf{B}_I^T \mathbf{q}(t), \quad \mathbf{I}_I^T \dot{\mathbf{p}}(t) = \mathbf{I}_I^T \mathbf{v}(t), \quad \mathbf{I}_I^T \dot{\mathbf{v}}(t) = \mathbf{I}_I^T \mathbf{a}(t) \\ \dot{\mathbf{b}}_a &= \mathbf{n}_{wa}, \quad \dot{\mathbf{b}}_g = \mathbf{n}_{wg} \end{aligned} \quad (5)$$

where $\mathbf{I}_I^T \mathbf{a}(t)$ is the UAV acceleration in the Inertial frame, ω is the rotational velocity in the UAV Body frame, $\mathbf{n}_{wa}$ and $\mathbf{n}_{wg}$ represent the white Gaussian noise that drives the IMU measurement drifts, and:

$$\boldsymbol{\Omega}(\omega) = \begin{bmatrix} -[\omega \times] & \omega \\ -\omega^T & 0 \end{bmatrix} \quad (6)$$

where:

$$[\omega \times] = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \quad (7)$$

2) *IMU Measurement Model*: The gyroscope and accelerometer measurements obtained by the IMU, in the Body frame B, are given by:

$$\begin{aligned} \boldsymbol{\omega}_{meas}(t) &= \boldsymbol{\omega}_{true}(t) + \mathbf{b}_g(t) + \mathbf{n}_g(t) \\ \mathbf{a}_{meas}(t) &= \mathbf{C} \left(\mathbf{B}_I^T \mathbf{q}(t) \right) \left(\mathbf{a}_{true}(t) - \mathbf{I}_I^T \mathbf{g} \right) + \mathbf{b}_a(t) + \mathbf{n}_a(t) \end{aligned} \quad (8)$$

where $C \left({}^B_I \mathbf{q}(t) \right)$ is the rotation matrix derived from the quaternion ${}^B_I \mathbf{q}(t)$, ${}^I \mathbf{g}$ is gravitational acceleration in the Inertial frame I, and $\mathbf{n}_g$ and $\mathbf{n}_a$ are zero-mean Gaussian white noise.

3) *Error Dynamics*: The error state of the system can thus be constructed as:

$$\tilde{\mathbf{x}} = \left[{}^B_I \tilde{\mathbf{q}}^T \quad {}^I \tilde{\mathbf{p}}^T \quad {}^I \tilde{\mathbf{v}}^T \quad \tilde{\mathbf{b}}_a^T \quad \tilde{\mathbf{b}}_g^T \right]^T \quad (9)$$

Consequently, the continuous-time error-state propagation equation is:

$$\dot{\tilde{\mathbf{x}}}(t) = \mathbf{F}_c(t) \tilde{\mathbf{x}}(t) + \mathbf{G}_c(t) \mathbf{n}(t) \quad (10)$$

where $\mathbf{n}(t)$ is the system noise, $\mathbf{F}_c(t)$ is the continuous-time error-state transition matrix and $\mathbf{G}_c(t)$ is the continuous-time input noise matrix, given by:

$$\mathbf{n}(t) = \left[\mathbf{n}_g^T \quad \mathbf{n}_{wg}^T \quad \mathbf{n}_a^T \quad \mathbf{n}_{wa}^T \right]^T \quad (11)$$

$$\mathbf{F}_c(t) = \begin{bmatrix} -[\dot{\omega} \times] & -\mathbf{I}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 \\ \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 \\ -\mathbf{C}^T \left({}^B_I \dot{\mathbf{q}} \right) [\dot{\alpha} \times] & \mathbf{0}_3 & \mathbf{0}_3 & -\mathbf{C}^T \left({}^B_I \dot{\mathbf{q}} \right) & \mathbf{0}_3 \\ \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 \\ \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{I}_3 & \mathbf{0}_3 & \mathbf{0}_3 \end{bmatrix} \quad (12)$$

$$\mathbf{G}_c(t) = \begin{bmatrix} -\mathbf{I}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 \\ -\mathbf{0}_3 & \mathbf{I}_3 & \mathbf{0}_3 & \mathbf{0}_3 \\ -\mathbf{I}_3 & \mathbf{0}_3 & -\mathbf{C}^T \left({}^B_I \dot{\mathbf{q}} \right) & \mathbf{0}_3 \\ -\mathbf{I}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{I}_3 \\ -\mathbf{I}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 \end{bmatrix} \quad (13)$$

B. Discrete Time Domain

The Kalman Filter presented in III-A can be discretised by taking Equations 5 and 12 and solving the following relation in order to find the discrete-time state-transition matrix, $\Phi(t_{k+1}, t_k)$, with initial conditions $\Phi(t_k, t_k) = \mathbf{I}_{16}$:

$$\dot{\Phi}(t_{k+1}, t_k) = \mathbf{F}_c(t_{k+1}) \Phi(t_{k+1}, t_k) \quad (14)$$

Furthermore, in the discrete-time implementation the noise propagation matrix is not included for obvious reasons, however, as this filter utilises inertial measurements these are included instead. Thus, the accelerometer and gyroscope measurements are modelled as inputs to our system:

$$\mathbf{u} = \left[\boldsymbol{\omega}^T \quad \mathbf{a}^T \right]^T \quad (15)$$

This leads to the following equations for the prediction step of the discrete-time filter:

$$\mathbf{F}_k = \begin{bmatrix} \mathbf{I}_4 + T_s \boldsymbol{\Omega}(\boldsymbol{\omega}) & \mathbf{0}_{3 \times 4} & \mathbf{0}_{3 \times 4} & \mathbf{0}_{3 \times 4} & \boldsymbol{\Omega}(\mathbf{b}_g) \\ \mathbf{0}_{3 \times 4} & \mathbf{I}_3 & T_s \mathbf{I}_3 & T_s^2 \mathbf{I}_3 & \mathbf{0}_3 \\ \mathbf{0}_{3 \times 4} & \mathbf{0}_3 & \mathbf{I}_3 & T_s \mathbf{I}_3 & \mathbf{0}_3 \\ \mathbf{0}_{3 \times 4} & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{I}_3 & \mathbf{0}_3 \\ \mathbf{0}_{3 \times 4} & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{I}_3 \end{bmatrix} \quad (16)$$

$$\mathbf{B}_k = \begin{bmatrix} \mathbf{0}_3 & \mathbf{0}_3 \\ \mathbf{0}_3 & \frac{1}{2} T_s^2 \mathbf{C} \left({}^B_I \mathbf{q}(t) \right) \\ \mathbf{0}_3 & T_s \mathbf{C} \left({}^B_I \mathbf{q}(t) \right) \\ \mathbf{0}_3 & \mathbf{0}_3 \\ \mathbf{0}_3 & \mathbf{0}_3 \end{bmatrix} \quad (17)$$

where T_s is the time interval of each discrete-time step, or, $T_s = t_{k+1} - t_k$.

This presented structure is the basis for a VINS and the method in which the position is measured is via the tracking of features in space. These can be included in the system in a variety of ways, one of which is the method presented in [4]. In this work the features themselves are included in the state vector as well as the camera-IMU calibration parameters. In such a setup the observation model, H_k , is as follows:

$$H_k = \frac{1}{\hat{z}_k^2} \begin{bmatrix} \hat{z}_k & 0 & -\hat{x}_k \\ 0 & \hat{z}_k & -\hat{y}_k \end{bmatrix} \mathbf{C} \left({}^B_I \mathbf{q}(t) \right) \dots \quad (18)$$

$$\begin{bmatrix} -[\mathbf{C} \left({}^B_I \mathbf{q}(t) \right) ({}^I \hat{\mathbf{p}}_f - \hat{\mathbf{p}}_k) \times] & \mathbf{0}_{3 \times 9} \dots \\ -\mathbf{C} \left({}^B_I \mathbf{q}(t) \right) & \mathbf{C} \left({}^B_I \mathbf{q}(t) \right) \end{bmatrix}$$

Such a framework can yield high usable results, however, as the objective of this work was not to implement a sparse Optical Flow-based navigation filter such a VINS has not been implemented for time and reliability reasons. As such this system is simulated by providing the aforementioned navigation filter with an orientation, position and velocity measurement taken from the ground truth with artificially added noise.

Thus, z_k and H_k are:

$$z_k = \begin{bmatrix} q_{sim.} \\ p_{sim} \\ v_{sim} \end{bmatrix}, H_k = \begin{bmatrix} \mathbf{I}_4 & \mathbf{0}_{4 \times 3} & \mathbf{0}_{4 \times 3} & \mathbf{0}_{4 \times 3} & \mathbf{0}_{4 \times 3} \\ \mathbf{0}_{3 \times 4} & \mathbf{I}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 \\ \mathbf{0}_{3 \times 4} & \mathbf{0}_3 & \mathbf{I}_3 & \mathbf{0}_3 & \mathbf{0}_3 \end{bmatrix} \quad (19)$$

where $q_{sim.}$, p_{sim} and v_{sim} are the simulated quaternion, position and velocity measurements, respectively.

IV. HIGH-FIDELITY SIMULATION ENVIRONMENT

The VINS presented in this work was tested in a high-fidelity simulation environment which will be explained in this section. Also, ROS was selected as the middleware framework to structure this system upon and the Hector Quadrotor package from Team Hector at Technische Universitt Darmstadt was selected as the UAV model to utilise. This UAV platform was modified to include six digital cameras aligned with each axis in order to implement the velocity estimation algorithm.

TABLE I: Simulated Camera Specifications

Camera Quantity	6
Resolution	320x320
FPS	10
H FOV	90 deg
VFOV	90 deg
Clip Distance (near)	0.1m
Clip Distance (far)	100m

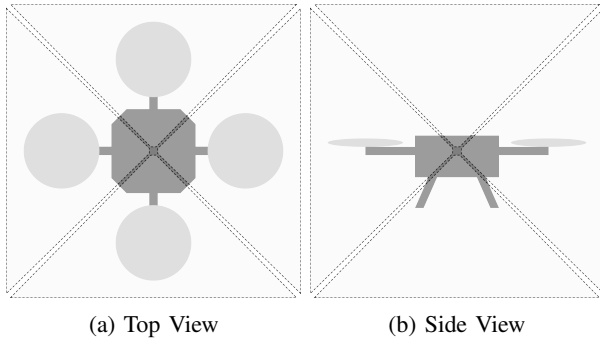


Fig. 3: Camera Configuration on UAV Model

Their orientations, shown in Figure 3 with the Field of View (FOV) shaded in grey, were selected in order to achieve omni-directional vision. Their specifications are presented in Table I. Furthermore, in order to test the work of this paper in an indoor environment, the Willow Garage model was chosen as it is a detailed model of an indoor office-like environment as well as being an open-source model that is easily accessible by anyone wishing to recreate this work. This model is shown in Figure 4. The optical flow algorithm utilised is the Farneback method [6] and is utilised instead of others for its computational speed; during development this was the only algorithm capable of parsing six video streams in real-time.

V. TESTS AND RESULTS

A. Velocity Estimation from Dense Optical Flow

1) *Monte Carlo simulation:* In order to test the velocity estimation algorithm a Monte Carlo approach was selected to investigate how a changing environment affected the velocity estimation. Two sets of tests were conducted, for both the circular and rectangular environments and their respective sizes were sampled randomly between minimum

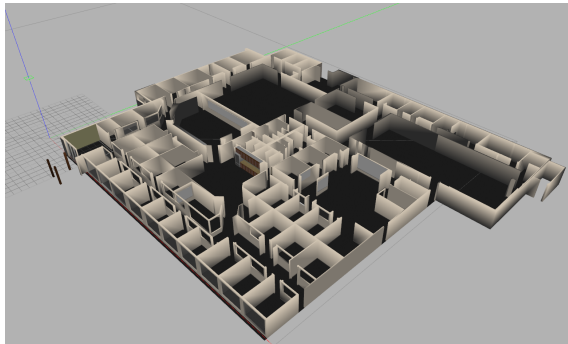
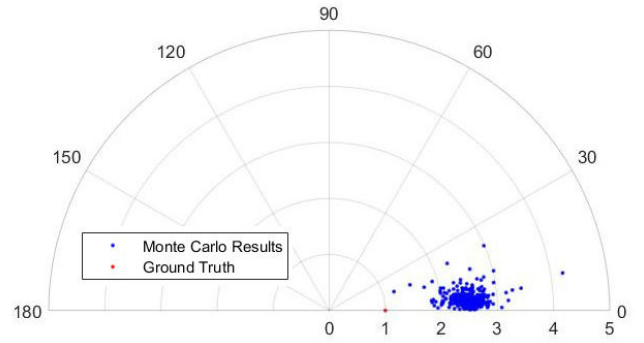


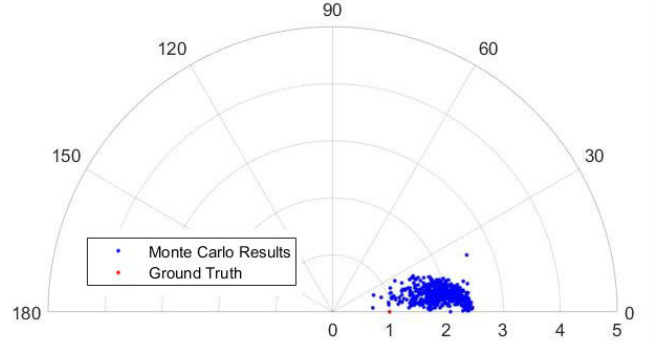
Fig. 4: Willow Garage Model

TABLE II: Parameters for Monte Carlo testing

Parameter	Minimum	Maximum	Units
Linear Velocity	$[-2 \ 2 \ 2]^T$	$[2 \ 2 \ 2]^T$	$[m/s]$
Angular Velocity	$[-2 \ 2 \ 2]^T$	$[2 \ 2 \ 2]^T$	$[m/s]$
Rectangular Env. Size	$[10 \ 3 \ 2]^T$	$[15 \ 6 \ 4]^T$	$[m]$
Circular Env. Size (radius)	3	6	$[m]$



(a) Circular Environment



(b) Rectangular Environment

Fig. 5: Processed Monte Carlo test results

and maximum values. Then the UAV velocity (both linear and angular) was randomly sampled between predefined limits, the Optical Flow patterns generated and the estimation algorithm run. The ranges for the relevant parameters of the test are summarised in Table II. For each of the two environments the Monte Carlo test was run 500 times and the output of each is an estimate of the UAV linear and angular velocity.

The error present between the estimation and ground truth was that of a scaling factor instead of a steady-state error. As such, in order to visualise the data all the results were normalised around the ground truth of that particular trail in order to clearly visualise the scaling error present. Then, the angular deviation between the estimate and ground truth vectors was calculated. These two values were then used to plot the results on a polar plot as shown in Figure 5. In Figure 5 the ground truth is indicated by a red marker and any perfect estimate would also lie at $[r, \theta] = [1, 0]$.

As can be seen by the results, shown with blue markers, the majority of the estimates lie centred on $[2.5, \pi/6]$ and $[2.5, \pi/6]$ for the circular and rectangular environments, respectively. The results are tabulated in Table III.

TABLE III: Monte Carlo test results

Environment	Scaling Error		Angular Error	
	Mean	Std. Dev.	Mean	Std. Dev.
Circular	2.52	0.20	5.70 deg	4.07 deg
Rectangular	1.94	0.30	14.43 deg	7.04 deg



Fig. 6: Optical Flow pattern (left) and Raw Image (right)

2) *Experimental Testing:* In order to test the Optical Flow algorithm experimentally the velocity estimation algorithm was applied to a live video feed from a connected webcam. For these tests a single camera was used for simplicity and as such the velocity estimation of a single axis was done. The camera was moved horizontally over a distance of approximately 10m manually and the camera was pointed at a feature rich wall to stimulate the optical flow pattern. A snapshot of the video feed and its corresponding optical flow pattern are shown in Figure 6.

The results for this test are shown in Figure 7 with the estimated velocity in blue. As the ground truth of this experiment is unknown it was estimated by filtering the raw signal, as shown in red. The filter in question was a 12th order low-pass IIR Butterworth filter with a half power frequency of 0.1Hz. The estimated velocity falls entirely within 0.2m/s of the ground truth and this estimate can be modelled as an error sampled from a normal distribution $\sim \mathcal{N}(0, 0.04)$ added to the ground truth.

In order to compare the Monte Carlo results with the experimental data the results from Section III were used to generate a simulated velocity measurement. Specifically, the noise values of these results were added to the estimated ground truth, the results of which can be compared against the experimental data, as shown in Figure 8. In this figure the estimated ground truth is in red, the experimental estimation in blue and the simulated estimation based on the MATLAB results in green. As can be seen the experimental estimation is an order of magnitude more accurate than the MATLAB equivalent, further showcasing the feasibility of the velocity estimation algorithm.

3) *Result and Discussion:* The results indicate that estimating the linear component of its velocity was possible and feasible. This was upheld when the assumption of a spherical environment was removed and it was shown that as the environment became more complex the estimate did not become unusable. As the standard deviation of the estimate was $\leq 10\%$ of the range of simulated velocities these results can be considered statistically significant. The results of the experimental test show how the horizontal velocity was estimated with high precision and could be considered as an added noise with a standard distribution of 0.04, making

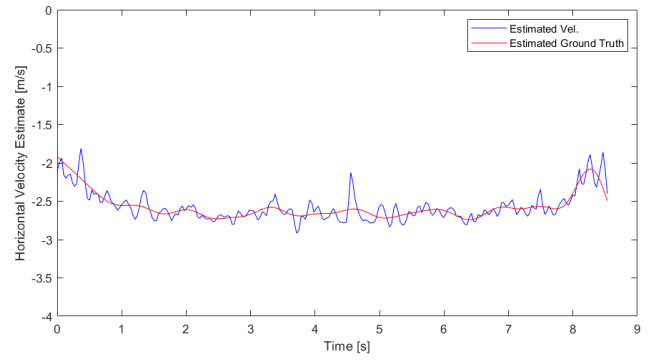


Fig. 7: Estimated Horizontal Velocity plot

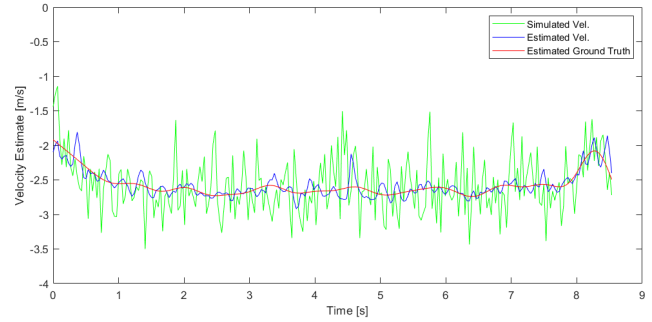


Fig. 8: Simulated and Experimental velocity estimation

this estimation statistically significant. The effectiveness of this algorithm supports the reasoning for the shortcoming of the simulation results, as the video used is of an outdoor environment with plenty of features for the optical flow algorithm.

B. VINS Filter Performance

1) *Test Methodology:* In order to test the VINS in a complete and representative manner the UAV model was made to perform a circular trajectory in the Willow Garage model. The accuracy of the EKF utilised can then be analysed to evaluate performance. With regards to the measurements provided to the EKF the IMU measurement originates from the simulated Hector package and has the following specification, summarised in Table IV.

The orientation, position and velocity measurements for the correction step of the filter were created in a similar manner by adding noise to the ground truth values. The velocity measurement was simulated instead of utilising the velocity estimation algorithm previously described due to the

TABLE IV: Simulated IMU specifications

Acceleration Drift	$[0.1 \ 0.1 \ 0.1]^T$	$[m/s]$
Acceleration Noise	$[0.35 \ 0.35 \ 0.35]^T$	$[m/s]$
Gyroscope Drift	$[0.1 \ 0.1 \ 0.1]^T$	$[rad/s]$
Gyroscope Noise	$[0.05 \ 0.05 \ 0.05]^T$	$[rad/s]$

erroneous estimates when implemented in Gazebo¹. For this reason the simulated measurements were created as follows:

$$\begin{aligned} q_{sim} &= q_{true} + q_{err} \\ p_{sim} &= p_{true} + p_{err} \\ v_{sim} &= v_{true} + v_{err} \end{aligned} \quad (20)$$

where q_{sim} , p_{sim} and v_{sim} are the simulated measurements, q_{true} , p_{true} and v_{true} are the ground truth values and q_{err} , p_{err} and v_{err} are the added noise signals for the orientation, position and velocity measurements, respectively. These noise signals are sampled as follows:

$$\begin{aligned} q_{err} &\sim \mathcal{N}(0, 0.05) \in \mathbb{R}^4 \\ p_{err} &\sim \mathcal{N}(0, 0.5) \in \mathbb{R}^3 \\ v_{err} &\sim \mathcal{N}(0, 0.04) \in \mathbb{R}^3 \end{aligned} \quad (21)$$

The standard deviation for the velocity noise measurement was selected from the experimental results and is therefore highly representative of the performance of the velocity estimation algorithm. Whereas the deviations for the orientation and position measurement were selected to be representative and chosen according to the performance from the following works [4], [7], [8]. The test performed is a clock-wise waypoint navigation of the trajectory shown in Figure 9 and covers a distance of approximately 60m through an office-like environment.

2) *Test Results:* The results of this test are shown in Figure 10. The plot shows the ground truth data in red and the filter state in blue. To analyse the performance of the filter the RSME error between the estimated state and the ground truth was calculated and is shown in Figure 11. The RSME plot shows that the filter is stable and that the

¹The smooth walls of the Willow Garage model failed to create useable Optical Flow patterns, therefore it was chosen to simulate this algorithm in a representative manner

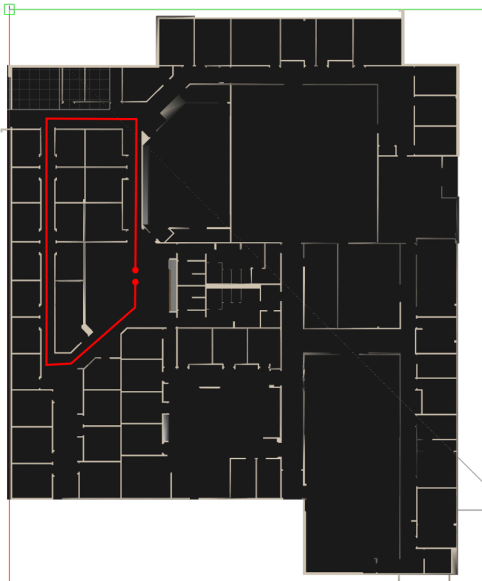


Fig. 9: Target Test Trajectory

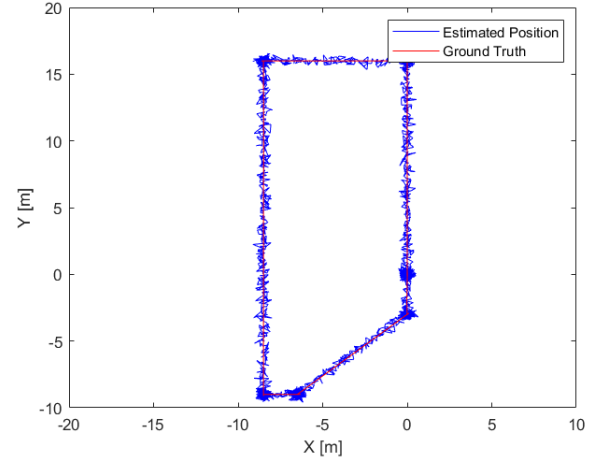


Fig. 10: Navigation Filter results

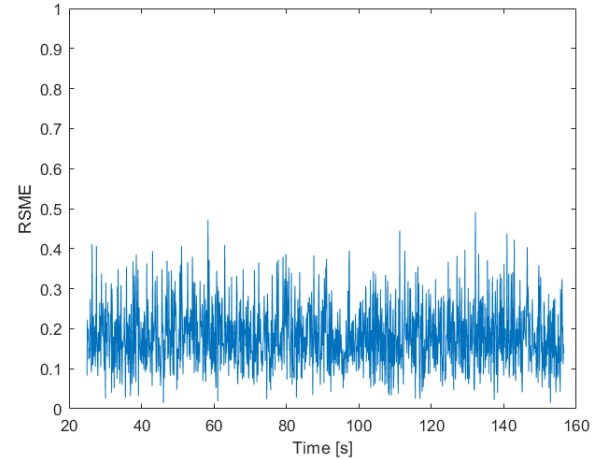


Fig. 11: Navigation Filter RSME

ground truth can be accurately tracked.

3) *Result and Discussion:* As implementing the velocity estimation algorithm in Gazebo led to erroneous data, this measurement was simulated according to the characteristics of the experimental results to maintain fidelity. The results of the VINS testing show that the state estimate of the EKF was able to accurately track the ground truth in the simulated environment with sufficient accuracy, as shown in the RSME results. This therefore shows that the VINS presented in this work is capable of accurately performing pose estimation of a UAV in an indoor environment.

C. Other Applications

Whilst this work, specifically the ROS framework, was developed with the goal of validating the idea of using dense optical flow for velocity estimation it can have applications in other works. The choice of utilising ROS for this work allows any part of this work to be ported to other application and platforms, for example a quad-rotor running a lightweight Linux kernel on a single-core processor. The

navigation filter can be applied to systems that do not utilise visual data for navigation and furthermore, as the VINS makes no assumption about its environment or the underlying platform dynamics it can be applied to other systems such as UGVs or UAVs.

VI. CONCLUSION

This paper presented a novel VINS approach that utilises dense omni-directional Optical Flow patterns to estimate velocity, primarily for a small UAV in an indoor environment. In order to test this system the velocity estimation algorithm was verified through a Monte Carlo simulation as well as experimentally. Then a high-fidelity simulation environment was developed in ROS and Gazebo, utilising a highly accurate UAV model to verify the VINS in an indoor office environment. All tests performed confirmed that this novel approach can be used for accurate navigation of a small UAV.

The next step for this work would be to implement this VINS in an on-board computer for a UAV, such as an ODroid UX4 [9]. This step would also involve implementing the sparse optical flow following the system design presented in [4], [7], [8]. This system could then be tested in a Vicon-equipped laboratory in order to experimentally measure the accuracy of the VINS.

ACKNOWLEDGMENT

This project is supported by Blue Bear Systems Research Ltd. The author would also like to thank Dr. Hyo-Sang Shin, Professor Antonios Tsourdos, Dr. Francis Salama and Mr. Innes Edridge for their support in this work.

REFERENCES

- [1] V. Tzitzilonis, K. Malandrakis, and L. Zanotti Fragonara, "Inspection of aircraft wing panels using unmanned aerial vehicles," *Sensors (Basel)*, 2019.
- [2] "Amazon prime air," <https://www.komando.com/happening-now/571255/look-up-in-the-sky-its-my-package-amazon-to-start-drone-delivery-within-months>, accessed: 2019-08-16.
- [3] "Optical Flow opencv documentation," <https://www.parrot.com/business-solutions-us/agriculture/inspection-and-analysis-crops>, accessed: 2019-08-16.
- [4] G. Huang, M. Kaess, and J. J. Leonard, "Towards consistent visual-inertial navigation," *IEEE International Conference on Robotics and Automation (ICRA)*, 2014.
- [5] N. Trawny and S. I. Roumeliotis, *Indirect Kalman Filter for 3D Attitude Estimation*, D. o. C. S. University of Minnesota and Eng., Eds., 2015.
- [6] "Optical Flow opencv documentation," https://docs.opencv.org/3.4/d4/dee/tutorial_optical_flow.html, accessed: 2019-08-06.
- [7] T. Lupton and S. Sukkarieh, "Visual-inertial-aided navigation for high-dynamic motion in built environments without initial conditions," *IEEE TRANSACTIONS ON ROBOTICS*, 2012.
- [8] C. Chen and H. Zhu, "Visual-inertial slam method based on optical flow in a gps-denied environment," *Industrial Robot*, 2018.
- [9] "Odroid ux4," <https://www.hardkernel.com/shop/odroid-xu4-special-price/>, accessed: 2019-08-04.